

Micro-service Interaction Analysis Using Alibaba Cloud Trace

Prasanna Kumar M, Rohit Bansal

Abstract

This project analyses micro service interactions using the Alibaba Cloud trace dataset to understand workload behaviour and service dependencies. A sample dataset is preprocessed and used to build a micro service interaction model. Key metrics such as latency and CPU utilisation are analysed to identify bottlenecks. Machine learning models are also explored for workload classification.

1. Introduction

In this project, we aim to analyse real-world micro service interactions using the Alibaba Cloud trace dataset, which provides detailed logs of service-to-service communication. The primary objective is to study the workload behaviour and understand the relationship and dependency between different micro services and system resources. By examining how services interact within a request (trace), we can model the overall system behaviour and identify patterns in execution.

The Problem: One of the major issue in the micro service level communication is the response time where for some particular type of requests, the response time from the requested service tends to be higher than the average response time. So our aim in this Project is to identify the actual cause for this delay in response time. A key focus of this project is to understand micro service interaction and response time based on different type of requests (http, db, mc, mq, rpc), furthermore to build a logistic regression model to classify whether a micro service request may take higher response time or not and notify the system so that a corresponding action can be taken to avoid overloading the node or to reduce the response time.

To perform the analysis we have used a small chunk of Alibaba dataset that contains 85164 services, running on 31490 nodes. Out of this we have randomly selected a sample consisting of 350000 records from the dataset, that contains 21475 services running on 24203.

2. Overview of Alibaba Micro service Trace

Alibaba handles billions of requests simultaneously using its large-scale, production-grade data center infrastructure, where the servers perform both online services and batch processing to optimize resource utilization. Each server is referred to as node in which multiple containers are created to handle requests and every type of requests runs on a dedicated server.

Alibaba Traces contain detailed runtime metrics of nearly twenty thousand microservices. They are collected from Alibaba production clusters of over ten thousand bare-metal nodes.

2.1 Cluster Architecture

The production cluster contains large number of bare-metal (BM) nodes and is running in the Alibaba Cloud. Users could submit their jobs through different controls, which requires resources from uniformed resource management (URM), or send micro services (MS) requirements to URM directly. The micro services and the offline jobs are scheduled on a separate containers to mitigate interference from offline jobs to provide performance guarantees for MS containers.

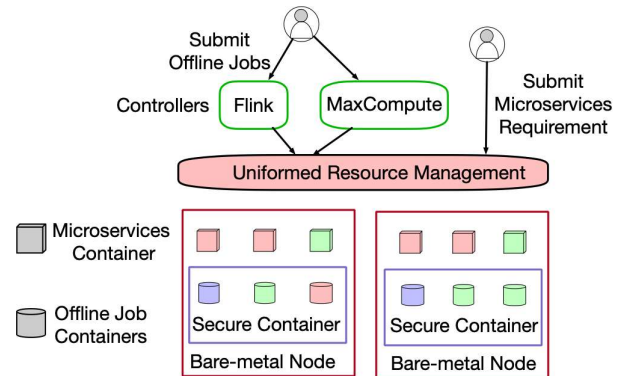


Figure 1-Cluster Architecture

2.2 MS architecture in Alibaba

We define the set of these calls as a call graph, which contains multiple calls between different pairs of micro services. Here, a pair of microservices contain one upstream microservice (UM) and one downstream microservice (DM).

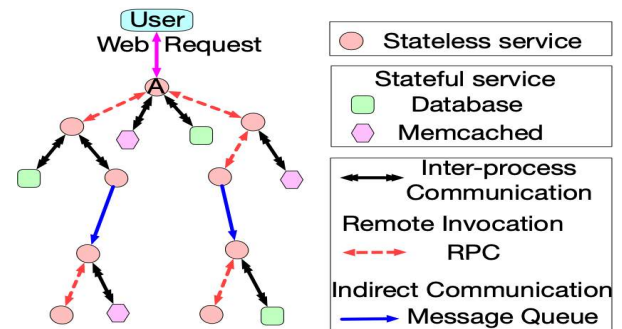


Figure 2-MS Architecture

Microservices can be categorized into two types, stateless services (e.g., a circle in the above Figure) and stateful services (e.g., a rectangle or hexagon). Stateless services are isolated from state data while stateful services need to store data in some locations, such as Database (DB) and Memcached (MC). There exist three types of communication paradigms between a pair of microservices, i.e., inter-process communication, remote invocation, and indirect communication.

Micro-service Interaction Analysis Using Alibaba Cloud Trace

Prasanna Kumar M, Rohit Bansal

3. Introduction of Trace Data

The traces include four parts of data as follows:

1. Node: BM Node runtime information. It records CPU and memory utilization of BM Nodes in a production cluster.

- **Timestamp:** Timestamp of recorded metrics. Range from 0 to 43200000 for twelve hours.
- **Nodeid:** The specific id of BM Node.
- **cpu_utilization:** CPU utilization of BM Node.
- **memory_utilization:** Memory utilization of BM node.

columns	Example Entry
timestamp	60000
nodeid	NODE_10632
cpu_utilization	0.7236219289416774
memory_utilization	0.738378997619996

Table 1-NODE_METRICS

2. MS_Metric_Table: It records CPU and memory utilization of every microservices along with the ndeid on which it is running

- **Timestamp:** Timestamp of recorded metrics.
- **msname:** The name of the microservice.MSName used here are stateless.
- **Msinstanceid:** The specific container id of MS.
- **cpu_utilization:** CPU utilization of MSInstanceID
- **memory_utilization:** Memory Utilization by Micro-service instance.

columns	Example Entry
timestamp	1000
msname	MS_21881
msinstanceid	MS_21881_POD_0
nodeid	NODE_11517
cpu_utilization	0.21995999999306
memory_utilization	0.83300145467122

Table 2-MS_METRICS

3. MSRTMCR_Table: It records the microservices response time for different operations.

- **Timestamp:** Timestamp of recorded metrics.
- **msname:** The name of the microservice.MSName used here are stateless.
- **Msinstanceid:** The specific container id of MS.

- **Metrics:** Calls Rate with different communications paradigms and corresponding RT (providerRPC_MCR,HTTP_MCR,providerMQ_MCR,consumerMQ_MCR,consumerRPC_RT).

- **Nodeid:** The specific id of BM Node.

columns	Example Entry
timestamp	60000
msname	MS_73317
msinstanceid	MS_73317_POD_1797
metrics	providerrpc_rt,providerrpc_mcr,consumerrpc_rt,consumerrpc_mcr,writemc_rt,writemc_mcr,readmc_rt,readmc_mcr,writedb_rt,writedb_mcr (all in milliseconds).
nodeid	NODE_3619

Table 3-MSRTMCT_METRICS

4. MS_CallGraph_Table: It records microservice to micro service interaction details along with response time.

- **Timestamp:** Timestamp of recorded metrics.
- **traceid:** Each call graph has a unique traceID.
- **rpcid:** Each call is identified by a unique rpcID,which contains the ID information of a pair of UM and DM.
- **UM:** The name of UM.
- **rpctype:** The communication paradigm. We record rpc_type as "DB" and "MC" for the calls via inter-process communication if DM is DB and MC respectively.
- **Interface:** The interface of DM is called by UM. The calls via remote invocation or HTTP have the interface.
- **DM:** The name of DM.
- **rt:** The response time of the Call. It is measured by millisecond.
- **Dminstanceid & uminstanceid:** The specific container id of dm MS and the specific container id of UM MS. An MS may have more than one container.

columns	Example Entry
timestamp	1153512
traceid	T_11560863075
service	S_153587416
rpc_id	0.1
um	MS_58845
uminstanceid	MS_58845_POD_0
rpctype	rpc

Micro-service Interaction Analysis Using Alibaba Cloud Trace

Prasanna Kumar M, Rohit Bansal

interface	xOuy6-80Vt
dm	MS_71712
dminstanceid	MS_71712_POD_244
rt	2

Table 4-CallGraph

4. Data Analysis

This is the first step in the process of identifying the cause of delay in response time. In here, we started first by understanding every table available and the relationship that can be extracted between the tables.

4.1 Data Collection

The dataset was downloaded using the bash command from the official Alibaba cluster data repository on github. [bash fetchData.sh start_date=0d0 end_date=1d1](#)

Above command downloads the zipped format of the 4 datasets after extracting, In each directory downloaded from above command perform:

[for file in ls *.tar.gz; do tar -xzf \\$file; done](#)

Above command Unzips the tar file and extracts the CSV file into current working directory.

4.2 Data Cleaning

After gathering the data, upon continued analysis we found some of the micro services in the dataset had NULL as the ID. As it was not useful for our analysis we went ahead with deleting of all the rows with unknown microserviceID. Furthermore the dataset was cleaned by removing NULL values.

4.3 Data Preparation

After download of csv file its was found that there are 85164 micro services data in CallGraph which require huge CPU and memory resource to analyze so out of 1.3Cr rows from the CallGraph 350,000 rows were randomly selected along with corresponding data from the other tables msmetric,mcrrt,node.

4.4 Data Interpretation

4.4.1 rpc_id interpretation

In this step, we started our analysis by understanding the significance and the meaning of values in the column are-

- **rpcid (CallGraph Table):** It is provided in the hierarchical manner starting from the root (0) and in every level the number is given that indicates the number of levels in the request

and in each level the numerical value indicate the micro service number, this micro service is making the request to

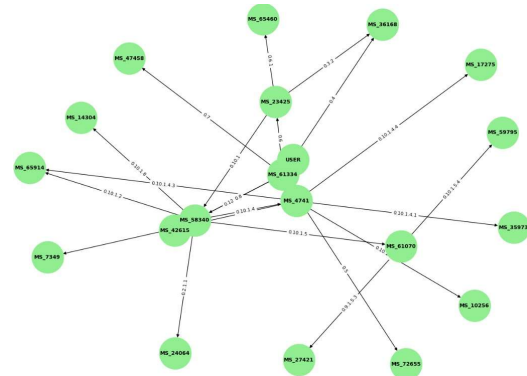
Ex: rpc_id = 0.3.2.4

0 is the **root call**, this is usually the entry point into the cluster.

0.3 is the **child of the root**, it indicates that the root is making call to the micro service whose value is 3.

4.4.2 Microservice CallGraph

The callgraph dataset contains the microservice response-request in an hierarchical manner represented by the rpc_id ,using which we were able to find out the [average callgraph length of a microservice that is 16.](#)



Micro-service Interaction Analysis Using Alibaba Cloud Trace

Prasanna Kumar M, Rohit Bansal

4.5 Key Findings

- Four different type of requests http, rpc, mc, mq, db
- Average Callgraph length is of 16
- USER is the top level request maker.
- Each service has multiple microservices.
- Database is the leaf call and it doesn't make call to any other node apart from sending the response.

5. Identification of Cause for High Response Time

After interpretation of the data ,went ahead to find the actual cause for higher response rate, this was done by sorting the data in the order of response time upon sorting the key finding is that the response time for http(USER) based request was higher than the other type of request.

So decode this further, the sample Callgraph file was grouped on the basis of type of request (http, rpc, mc, mq) and average response time of each type of request was plotted.

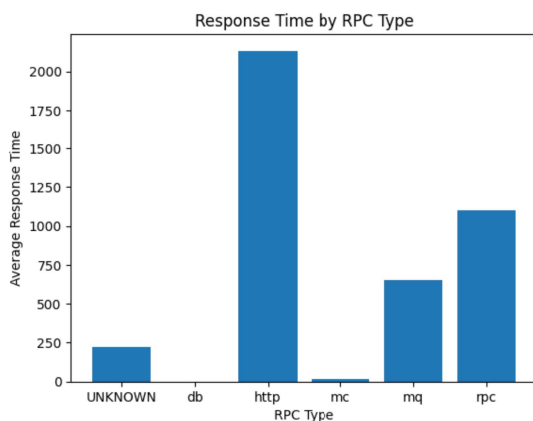


Figure 4-Request Type V/s Avg Response Time

From the above image, it can be concluded that the average response time for the http based request was found to be higher than the rest. So for further analysis the data was trimmed down and only rows that were making http request were selected.

5.1 Initial Methodology

Identifying CPU utilization and memory utilization of and comparing it with the response time for micro services from all type of requests to identify whether the resource utilization of micro-service is the reason for higher response rate.

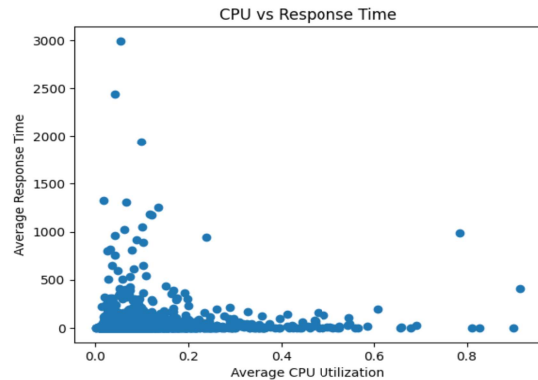


Figure 5-CPU Utilization v/s Response Time

5.1.1 CPU utilization v/s Microservices

A simple CPU utilization of micro services were plotted to understand whether CPU utilization is the cause for higher response rate. But as represented by the above figure the plot is showing no increase in its value as the response time increase so it was concluded that CPU utilization alone can't be the reason for higher response time.

5.1.2 Memory utilization v/s Microservices

A simple CPU utilization of micro services were plotted to understand whether CPU utilization is the cause for higher response rate. But as represented by the above figure the plot is showing no increase in its value as the response time increase so it was concluded that CPU utilization alone can't be the reason for higher response time.

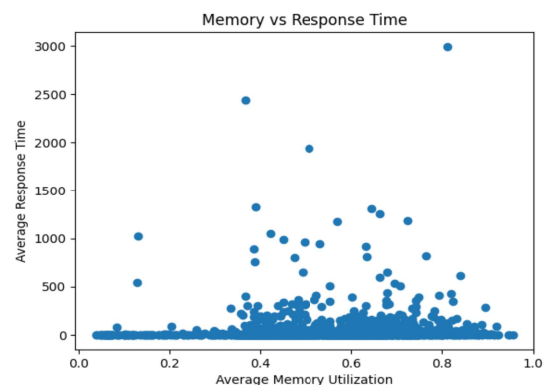


Figure 6-Memory Utilization v/s Response Time

5.2 Methodology

Calculating the average response time for every trace id that combines all the micro services that it has and plotting it in bar graph and plotting a normalized CPU utilization value, memory utilization value and selecting a micro services running on high contention node and plotting its contention in line graph to identify which of the above three closely resemble the bar plot.

Micro-service Interaction Analysis Using Alibaba Cloud Trace

Prasanna Kumar M, Rohit Bansal

- Unlike the initial methodology, here we selected the http request data and grouped that dataset by traceid.
- Calculated the average response time, average cpu_utilisation and memory_utilisation for each of the trace_id.
- Calculated the number of microservices contending for a node and considered it as the node contention value.
- Performed join on the initial http selected dataset later on after grouping the data based on traceid, we identified the node that is having higher contention value for each traceid.
- Plotted the average response time using bar graph.
- Rest of the metrics average cpu_utilization, memory utilization and node contention were plot as the line plot in the same graph.
- The line plot that actually resembles the bar plot can be concluded as the cause of higher response time.

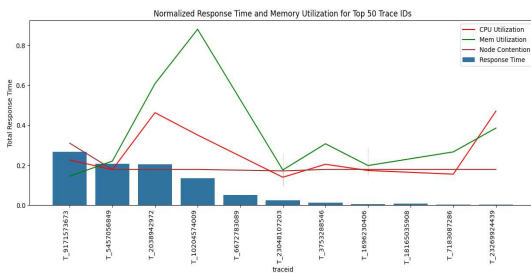


Figure 7-Response Time V/s Metrics

The above was plotted for top 15 traceids with high response time, in the figure green line plot is for memory utilisation, red line plot represents cpu_utilization and the brown line plot represents the node contention.

From the above figure, it can be concluded that CPU utilization solely has no role in increasing the response rate but memory utilization resembles about 40% of the response time plot. Out of all three node contention plot is the one which resembles the response time when compared with other two.

5.3 Key Finding

So from above observations, we can conclude that node contention is first major cause for higher response time followed by memory utilization.

6. Solution for Higher Response Time

As from our findings we observed that node contention is the major cause for increase in the average response time.

We came with a solution to build a logistic regression model on the CallGraph dataset to identify the

microservice requests that may request a service that can run in the high contention node.

6.1 Data Preparation

Initially, the callgraph dataset was trimmed and only the required columns were selected for model training. Furthermore, the data from the msmetric was grouped by nodeid and a new column called total_calls was created that represents number of microservices calling the node(calling_details).

The calling_details data was joined with the ms_metric on 'nodeid' resulting in the dataset containing columns timestamp, msname, msinstanceid, nodeid, cpu_utilization, memory utilization, total_calls. Out of which only the msname, msinstanceid, nodeid, total_calls were selected.

The resulting df_joined_call_graph data was joined with original call_graph table on call_graph('um', 'uninstanceid'), df_joined_ms_metric('msname', 'msinstanceid').

As for the training we only need columns that represent the request, the joined call_graph table was trimmed and unnecessary columns were removed and also the duplicates value were removed.

Final Training Data

CallGraph('rpc_id', 'rpctype', 'um', 'uninstanceid', 'interface', 'dm', 'dminstanceid')

6.2 Data Encoding

Assigned a unique random integer to each category of value of every column.

6.3 Data Enhancement

Building a binary column to indicate whether the micro service is requesting a service that runs on a high contention node or not.(is_high_request_node).

To achieve this, we classified top 25% of the node based on the contention value as high contention node and rest as low contention node.

6.4 Model Building

The data was split into training (60%) and testing (40%). StandardScaler is applied on data, so each feature has a mean of 0 and a standard deviation of 1. Model was built on this training data 'liblinear' function was used to find the weights with a maximum iteration of 1000 to find the optimal solution.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
log_model = LogisticRegression(solver='liblinear',
max_iter=1000, class_weight='balanced')
log_model.fit(X_train_scaled, y_train)
```

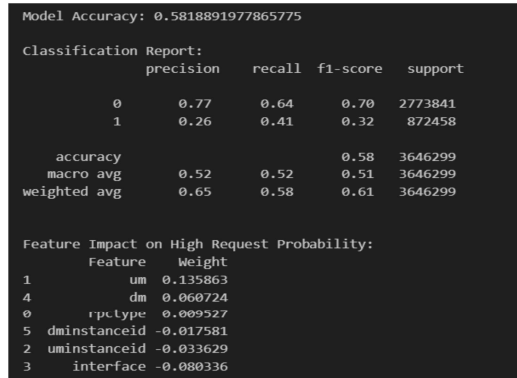

Micro-service Interaction Analysis Using Alibaba Cloud Trace

Prasanna Kumar M, Rohit Bansal

6.5 Results

The Model trained using a 60L rows dataset after which the model achieved a overall accuracy of around 58.6%.

The columns that are used to train model has a significant importance over model accuracy.



```
Model Accuracy: 0.5818891977865775
Classification Report:
      precision    recall  f1-score   support
0           0.77       0.64       0.70       2773841
1           0.26       0.41       0.32        872458

 accuracy          0.58       3646299
 macro avg          0.52       3646299
weighted avg          0.65       3646299

Feature Impact on High Request Probability:
Feature      Weight
1          um    0.135863
4          dm    0.060724
0          rptype 0.009527
5  dminstanceid -0.017581
2  uminstanceid -0.033629
3          interface -0.080336
```

Figure 8- Accuracy Result

The recall shows that the model can clearly identify nearly half of high node requests which is significant considering the size of data.

From the Feature Importance we can say that the micro service which is sending request and the one which is receiving the request dominate in the model training.

7. Conclusion

In this project, we aimed to solve the high response rate problem by identifying the real cause (node contention) and by utilising the Logistic Regression and preprocessing techniques, we were able to achieve 58.6% accuracy in identifying the requests that may access services requiring high contention node.

These results show that through analysis and ML technique the requests that may cause node traffic is predictable.

8. Future Work

We'll continue to explore the improve the model either by further analysis of data or by feature engineering.

The above analysis and the model can be used to identify the requests that may trigger node traffic and further actions like-

- Suspending this request temporarily based on the node container availability to ease the traffic can be done.
- Caching of data (static or dynamic) that resulted in higher response time and, if the same request comes again then serving the cached data directly instead of allocating the containers can be done.
- Alerting the System/employee to take further actions when continuous high number of request comes to high contention node can be done.

9. References

[1] Cluster Architecture

<https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2022>

[2] Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis

<https://dl.acm.org/doi/10.1145/3472883.3487003>